

RICORSIONE
Forma di programmazione alternativa all' iterazione

Funzione/procedura ricorsiva → Una funzione si dice ricorsiva se richiama se stessa

```
int fatt (int n) {  
    if (n==0)  
        return 1;  
    else  
        return n * fatt(n-1);  
}
```

informatico

non e' una chiamata in coda

matematico

ricorsivo

$f(n) = f(n-1) * n$

$f(0) = 1$

NO induttivo

```
int fie (int n) {  
    if (n==0)  
        return fie(0);  
    }  
}
```

```
{  
    foo(0) = 0;  
    foo(n) = foo(n+1);  
}
```

no induttivo

```
int foo (int n) {  
    if (n==0) return 0;  
    else return foo(n+1);  
}
```

ricorsivo

Perche' un linguaggio permetta la ricorsione

- ↳ Procedure nel linguaggio
- ↳ Gestione dinamica della memoria

Ricorsione in coda → forma ottimizzata (rispetto all'uso della memoria) della ricorsione

Una chiamata dentro f ad una procedura g si dice in coda se f restituisce il valore ritornato da g senza ulteriori computazioni:

Una procedura si dice ricorsiva in coda se la chiamata ricorsiva e' in coda

```
int fatt (n) {  
    return fattrec(n, 1);  
}  
  
int fattrec (int n, int res) {  
    if (n==0) return res;  
    else return fattrec(n-1, res*n);  
}
```

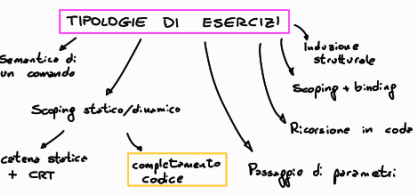
```
Fib(0) = 1  
Fib(1) = 1  
Fib(n) = Fib(n-2) + Fib(n-1)
```

```
int Fib (int n) {  
    if (n <= 1) return 1;  
    else return Fib(n-1) + Fib(n-2);  
}
```

NON E' IN CODA!

in coda

```
int fib(n) { return fibrec(n, 1, 1); }  
int fibrec(int n, int res1, int res2) {  
    if (n==0) return res1;  
    else return fibrec(n-1, res2, res1+res2);  
}
```



Esercizio di scoping di completamento del codice

```
{  
    int i;  
    ② for (int i=0; i<=L; i++) {  
        int x;  
        x = fun(i);  
    }  
}
```

- Fornire il codice da inserire in ② in modo tale che
1. Se il linguaggio ha scoping statico le due chiamate a fun restituiscono lo stesso valore
 2. Se il linguaggio ha scoping dinamico le due chiamate a fun restituiscono valori diversi

→ il codice deve essere eseguibile → dobbiamo definire `fun()`

⇒ Perché il comportamento di un programma sia potenzialmente diverso indipendente dalle regole di scoping è **NECESSARIO** che ci sia:

- una procedura con ambiente non locale
- il riferimento non locale deve esistere sia nel contesto di definizione che in quello di chiamata

→ i è un buon candidato per essere ambiente non locale per fun
x non è inizializzata nell'ambiente di chiamata
→ non si può usare

④ $\left\{ \begin{array}{l} i=1; \text{ --- serve per rendere il codice eseguibile} \\ \text{int fun() \{ return i; \}} \end{array} \right.$

Esecuzione per scoping statico

↳ la chiamata di `fun()` restituisce la `i` globale che vale 1 e non viene modificata

⇒ restituisce 1 ad entrambe le chiamate

Esecuzione con scoping dinamico

↳ le due chiamate di `fun()` fanno riferimento alla `i` del ciclo for che cambia tra una chiamata e l'altra
prima chiamata → return 0
seconda chiamata → return 1

Esercizio 2

```
{ int x;
  ④
  void exec() {
    for (int j=2; j<=3; j++) {
      int y=t;
      x=fun();
    }
  }
  exec()
}
```

codice t.c. `fun()` restituisce lo stesso valore nelle due chiamate con scoping statico;
restituisce valori diversi con scoping dinamico

→ Dobbiamo definire `fun()` con ambiente non locale che possa dipendere dalle regole di scoping

↳ uniche variabili candidate sono `y` e `j` perché l'ambiente di chiamata non è modificabile tra le due `j` cambia ad ogni esecuzione quindi permetterebbe di rispettare la richiesta

⇒ Dobbiamo definire anche `j` globale che non viene modificata

⇒ ④ $\left\{ \begin{array}{l} \text{int j=1} \\ \text{int fun() \{ return j; \}} \end{array} \right.$

scoping statico / scoping dinamico

La chiamata ad ogni iterazione restituisce la `j` globale che vale 1
La chiamata restituisce la `j` del ciclo for che vale 2 alla prima iterazione e 3 alla seconda

Esercizio 3

```
int a=0;
  ④
  while (a<=1) {
    int x;
    ④
    x=fun();
    a++;
  }
```

richiesta come es. precedente

ipotesiamo come nel primo esercizio che il blocco del while possa definire un ambiente locale separato da quello globale

Definiamo `fun()` per renderla eseguibile e `fun()` deve contenere un ambiente non locale per dipendere dallo scoping

④ $\left\{ \begin{array}{l} \text{int x=2;} \\ \text{int fun() \{ return x; \}} \end{array} \right.$
scoping statico / scoping dinamico
L'ambiente non locale per `fun` è quello globale quindi le due chiamate restituiscono 2 (valore di `x` non modificato)
x non è inizializzata → la funzione in ④
x=0

La prima chiamata restituisce `x=0`
La seconda `x=0+1`

Fore:

```
{ int a=0; int y=1;
  ④
  void exec() { int x;
    ④
    y=fun();
    a++;
  }
  while (a<=1) { exec(); }
```

richiesta esercizio precedente